

DROS：自主 AI 軟體系統中確定性權能強制之執行期架構

陳濬程 Chun-Cheng (Jimmy) Chen
康宸園有限公司 (Top-Celestial Company Ltd.)
中華民國臺灣臺北 (Taipei, Taiwan R.O.C.)
jimmychen@dr-os.io

Abstract—自主 AI 軟體系統日益在共享的應用程式執行期 (Runtime) 內並發執行多個邏輯代理人、角色與工作流。儘管應用層策略機制能夠區分這些邏輯主體，但傳統的作業系統存取控制通常僅能觀測到外層包裹的進程識別項 (PID/UID)，無法識別對特定工具調用負有責任的代理人身分。此種不匹配在軟體層授權決策與底層執行強制之間造成了嚴重的歸因鴻溝。

本文提出 DROS——一種能夠跨越「應用程式至執行邊界」保持代理人層級歸因的四層執行期架構，並在受管轄操作進入其具體實作前實施確定性權能強制。DROS 結合了語義過濾、密碼學認證的執行身分、拓撲感知之屬性授權，以及基於不可變角色專屬權能點陣圖的 C-ABI/FFI 強制核心。該架構將機率性的上游決策機制與確定性的執行邊界解耦，使後者能夠提供獨立於語義分類準確性的條件執行保證。

我們在涵蓋 160,611 次請求的 24 小時評測中驗證了 DROS，其中包括橫跨提示注入、目標劫持、特權提升與供應鏈攻擊模式的 137,751 次對抗性嘗試。在定義的受管轄執行期邊界下，DROS 對於到達強制邊界的對抗性調用達到了觀測上的 100% 阻斷率，且在 22,860 筆正常請求中的策略誤報率為 0/22,860。下游策略路徑的中位延遲為 26.21 μ s (P99 延遲為 242.69 μ s)，而獨立的權能檢查平均僅需 825 ns。最小基質極限評測進一步證實在 906 次敵意嘗試中觀察到零未授權權能轉移。評測結果證實：確定性執行強制可作為自主 AI 軟體系統的一項內建執行期架構特性，而無需單純依賴上游的機率語義防禦。

Index Terms—軟體架構、自主 AI 系統、執行期強制、權能安全、屬性存取控制、執行監控、實證軟體工程。

I. 引言 (INTRODUCTION)

主 AI 軟體系統正日益結合大型語言模型 (LLM) 與外部工具、服務、記憶體系統、軟體函式庫及執行環境。這種架構賦予代理人以極少人類干預完成多步驟任務的能力，但也引入了傳統進程層級授權機制無法充分解決的軟體工程難題。

現代代理人執行期常在單一作業系統進程內託管多個邏輯主體。這些主體可能代表不同的代理人、角色、工作流或被委託之任務，同時共享相同的進程身分、位址空間與執行期基礎設施。因此，作業系統授權機制可能正確判定某個進程被允許存取某項資源，卻完全無法判定負責當前具體操作的特定邏輯代理人是否具備該行動的授權。

相對地，應用層策略系統雖具備豐富的情境資訊 (能識別代理人、角色、任務、工作流與工具狀態)，但其決策唯有在每項安全相關操作均穿透相應的策略強制點時方能生效。這在軟體層歸因與執行層強制之間形成了架構性的落差，我們將此定義為**代理人-至-執行歸因鴻溝 (Agent-to-Execution Attribution Gap)**：

定義 1 (歸因鴻溝, Attribution Gap)：在共享的執行期身分下，應用層代理人主體與執行層授權決策之間缺乏值得信賴且可驗證的映射關係。

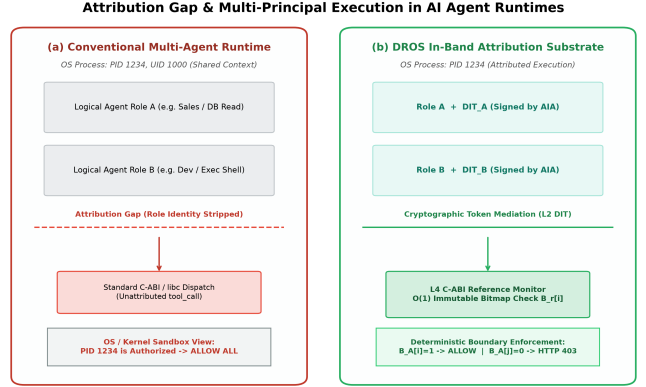


Fig. 1. 多主體代理人執行期中的代理人-至-執行歸因鴻溝與 DROS 帶內歸因基質。

當兩個或更多邏輯上不同的授權主體在同一個作業系統安全主體 (如共享的 PID/UID) 內執行，而底層執行基質在強制邊界上無法以密碼學方式區分其授權狀態時，即存在歸因鴻溝。

本問題不等同於傳統的「困惑代理人 (Confused Deputy)」行為 [1]。在經典困惑代理人情境中，特權組件受騙代表非預期請求者行使其特權；而在本研究情境中，更根本的挑戰在於：執行期在「計算授權」與「執行請求操作」這兩個斷點之間，丟失了邏輯主體的身分歸因。

當單一進程託管具備不同權能包絡線的多個代理人時，此區分尤為關鍵。例如，給定包含三個邏輯角色的執行環境 $\mathcal{R} = \{r_1, r_2, r_3\}$ ，其中 r_1 允許存取資料庫， r_2 允許調用外部網路，而 r_3 均不允許。若三者運行於同一進程內，傳統進程控制僅能觀察到 $PID = P$ ，而無法分辨 $(P, r_1), (P, r_2), (P, r_3)$ 。

如圖 1 所示，這種非對稱性形成了歸因鴻溝：作業系統沙箱僅能看到單一旦經授權的宿主進程 PID，而執行期在分派時遺失了邏輯角色身分。DROS 引入了帶內歸因基質，將每次調用綁定至經認證的密碼學身分令牌，使下游參考監視器能夠強制實施角色專屬的權能邊界。

核心論題 (Core Thesis)：一項先前未授權之執行，絕不應依賴語義決策維持正確才能被阻斷。DROS 建立執行邊界參考監視器，確保代理人規劃層的認知失陷本身絕不會將其執行權限擴展至超出其密碼學綁定的能力包絡線之外。

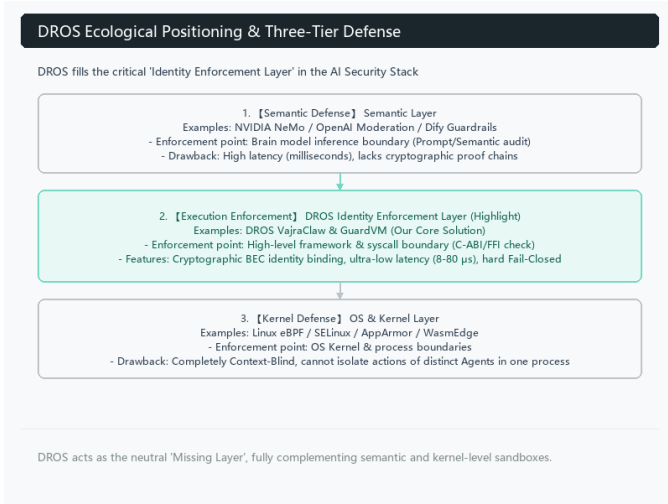


Fig. 2. DROS 控制平面與執行期強制平面架構。

A. 本文貢獻 (Contributions)

- 1) **問題定位與形式化**：界定並形式化多租戶代理人軟體系統中的歸因鴻溝。
- 2) **架構機制**：提出 DROS 四層執行期架構，將機率語義分析與確定性二進位能力邊界解耦。
- 3) **形式化不變式**：在顯式運作公理下證明權能強制不變式 (CEI)。
- 4) **實證系統評測**：基於 24 小時 160,611 次請求展開全面的防禦力、消融實驗、延遲開銷與邊界覆蓋評測。

II. 背景與威脅模型 (BACKGROUND & THREAT MODEL)

A. 代理人攻擊向量矩陣

我們將當前自主代理人系統面臨的威脅分類彙整如表 I。

B. 對手模型 (Adversary Model)

我們評估在以下條件下運作之對手：

- 對手可透過外部管道輸入任意對抗數據；
- 對手可發起多輪對話操縱與間接提示注入；
- 對手完全知悉上游語意過濾啟發式規則；
- 對手**無法**在無私鑰 K_{AIA} 下偽造密碼學簽名；
- 對手**無法**直接修改防寫記憶體頁面或竄改進程內已編譯的二進位程式碼；
- 對手局限於應用執行期內，**無法**直接攻陷作業系統核心或虛擬機監視器。

III. DROS 執行期架構 (DROS RUNTIME ARCHITECTURE)

DROS 由四個解耦層級構成，形成從機率語義推理到確定性二進位執行的架構流水線（如圖 2 所示）。

A. 第一層：語義邊界層 (Semantic Boundary)

L1 負責對輸入提示詞與工具調用情境進行前置語意檢查，定位為機率性分類過濾而非確定性授權。在評測語料庫中，L1 過濾了 85.2% 的已知模板 IPI，對混淆攻擊阻擋率為 0%；正常請求誤報率為 0/22,860。其失效模式完全交由 L2-L4 處置。

B. 第二層：密碼學執行身分層 (Cryptographic Execution Identity)

L2 引入 DROS 身分令牌 (DIT)：

```
DIT := {
  agent_id:    UUID,
  role_id:     uint32,
  task_id:     UUID,
  bitmap_hash: SHA-256(B_r), // 工作流審計與溯源綁定
  issued_at:   Unix timestamp, // 策略版本同步之密碼學承諾
  expires_at:  Unix timestamp // 短 TTL <= 15 分鐘,
  signature:   Ed25519(private_key_AIA, DIT_body)
}
```

權威來源與密碼學承諾解耦：DIT 內的 `bitmap_hash` 僅作為密碼學承諾，確保代理人與 GuardVM 參照完全相同的編譯策略紀元；真正的授權權威嚴格源自 GuardVM 防寫頁面中的 $B_r = \text{PolicyStore}[r]$ 。DIT 必須由獨立信任域的自動簽發機構 (AIA) 簽發，受入侵的代理人進程無法自行鑄造或偽造未授權角色。

C. 第三層：執行期拓撲與屬性強制層 (Runtime Topology & ABAC)

L3 施加多代理人拓撲與情境屬性約束（有向無環溝通圖、跨部門調用邊界、工作流調用序列、扇出上限及符號化 ABAC 規則 [15]）。

D. 第四層：確定性執行邊界層 (Deterministic Execution Boundary)

L4 為核心執行機制。每個受管轄操作映射至固定索引 $m_i \rightarrow i$ 。對於經認證角色 r ，強制操作為 $\text{allow} = B_r[i]$ 。決策時間相對於系統工具總數恆為常數 ($O(1)$)。

完整執行序列為：

$$\begin{aligned} \text{Verify(DIT)} &\rightarrow \text{Resolve}(r) \rightarrow \text{Load}(B_r) \\ &\rightarrow \text{Check}(B_r[i]) \rightarrow \text{Execute}(m_i) \end{aligned} \quad (1)$$

若 $B_r[i] = 0$ ，分派器返回 HTTP 403 授權錯誤，絕不進入 m_i 的實作主體。

IV. 策略狀態與並發熱更新 (POLICY STATE & HOT UPDATES)

為確保策略狀態無法被受管轄代碼竄改，每個角色的 B_r 作為不可變策略快照載入於防寫記憶體頁面中。策略更新使用 RCU（讀-複製-更新）原子指標交換進行無鎖置換：

$$B_r \xrightarrow{\text{atomic}} B'_r \text{ 而非原地寫入 } B_r[i] \leftarrow x \quad (2)$$

圖 3 展示了此雙緩衝 RCU 指針交換協議，確保傳播延遲 $< 1.0 \mu\text{s}$ 且不在熱路徑上引入全域互斥鎖。

V. 安全模型與形式化性質 (FORMAL SECURITY MODEL)

A. 運作公理 (Operational Assumptions)

- **假設 A1 (邊界完全仲裁)**：所有安全相關操作 $m \in \mathcal{M}$ 均嚴格經由插樁之 DROS C-ABI 介面分派。
- **假設 A2 (密碼學不可偽造性)**：對手在無私鑰 K_{AIA} 下無法偽造 Ed25519 簽名，且 SHA-256 碰撞在計算上不可行。
- **假設 A3 (策略不可變性)**：已編譯之能力庫 B 被載入進程內託管代碼無法寫入的防寫記憶體頁面中。
- **假設 A4 (分派完整性)**：L4 參考監視器在移交控制權予工具主體前同步完成授權評估。

TABLE I
代理人攻擊向量矩陣 (AGENTIC ATTACK VECTORS MATRIX)

攻擊家族	MITRE ATLAS	攻擊機制	目標資產
間接提示注入 (IPI)	AML.T0051	攻擊者在代理人讀取的外部非受信任數據 (郵件、PDF、資料庫) 中嵌入惡意指令	工具調用控制流
目標劫持	AML.T0054	上下文污染竊改代理人的長遠目標，誘發未授權的多步行動鏈	代理人規劃狀態
特權提升	AML.T0053	受侵害代理人利用有效憑證調用高權限端點 (如 <code>deploy_prod</code> 、 <code>bulk_delete</code>)	授權邊界
供應鏈污染	AML.T0010	受污染的模型權重或數據集使攻擊者能夠橫向移動至下游企業系統	跨組織信任邊界

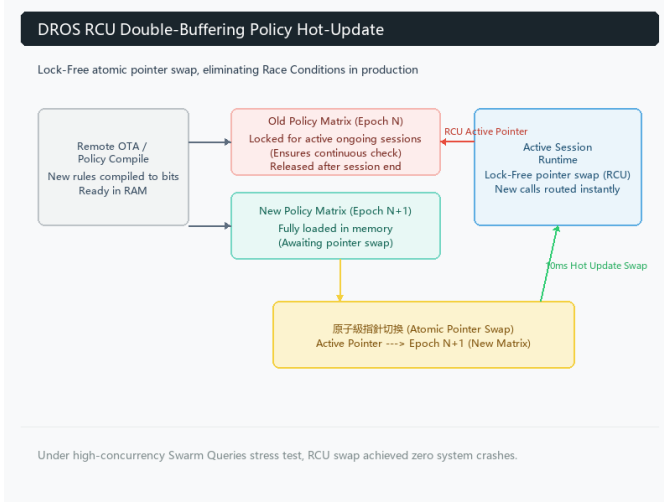


Fig. 3. DROS GuardVM 中無鎖 RCU 雙緩衝策略熱更新機制。

B. 形式化執行定理

令 $\text{GExec}(r, m_i, s)$ 代表工具 m_i 代表狀態 s 下的角色 r ，經由已登記之 DROS 分派路徑執行完成。

定理 1 (確定性執行遏制): 在假設 A1–A4 下，對於所有代理人角色 $r \in \mathcal{R}$ 、登記工具 $m_i \in \mathcal{M}$ 及執行狀態 $s \in \mathcal{S}$ ，若 $B_r[i] = 0$ ，則未授權執行絕不可能發生：

$$B_r[i] = 0 \implies \neg \text{GExec}(r, m_i, s) \quad (3)$$

Proof: 由假設 A1，執行 m_i 必須進入 DROS C-ABI 參考監視器。由假設 A4，僅當監視器返回 ALLOW 時方進入實作主體。依 L4 構造，主體嚴格綁定自經認證之 DIT：

$$\text{DispatchPrincipal}(x) = \text{AuthenticatedRole}(\text{DIT}) = r \quad (4)$$

由假設 A2，無私鑰無法偽造角色 r 之有效令牌；由假設 A3，點陣圖於執行期防寫不可逆。因此當 $B_r[i] = 0$ 時，比對恆為偽，立即觸發 FFI panic 終止分派。故 $\text{GExec}(r, m_i, s)$ 恆為偽。 ■

性質 1 (爆炸半徑不變式): 令 $\mathcal{C}^* = \text{Hostile}$ 代表代理人規劃狀態之全面失陷。在假設 A1–A4 下：

$$\mathcal{C}^* = \text{Hostile} \wedge \text{定理 1} \implies \mathcal{E}_{\text{effective}} \subseteq \mathcal{B}_{r_{\text{assigned}}} \quad (5)$$

即：代理人的認知失陷絕不會擴展其超出預先承諾權能包絡線之外的執行權限。

VI. 實作與實證評測 (EMPIRICAL EVALUATION)

A. 實作與測試環境

核心由 Rust 實作 (`libdros.so` / `dros.dll`)，並以 C-ABI 暴露給 Python 3.11 及 LangChain 執行期。測試硬體為 Intel Xeon E3-1275L v3 @ 2.70 GHz · Ubuntu 22.04 LTS 及 Windows 10 IoT Enterprise LTSC。開源評測庫存放於：<https://github.com/Top-Celestial-Company-Ltd/DROS-VEP-lite>。

B. 評測方法論

24 小時內共評估 160,611 次請求，包含 22,860 筆正常請求與 137,751 筆橫跨四大攻擊家族的惡意負載 (已知模板 IPI: 68,420；對抗混淆 IPI: 52,180；目標劫持: 9,251；特權提升: 6,112；供應鏈污染: 1,788)。

C. 消融實驗結果 (RQ2)

消融實驗結果如表 II 所示。

核心發現：對於已知模板 IPI，上游層 (L1–L3) 過濾了 93.5%，存在 6.5 個百分點落差。第四層 (L4) 消除了這最後 6.5 個百分點的殘餘風險，並阻斷了 100% 完全穿透 L1–L3 的 52,180 筆對抗混淆負載。

D. 效能與延遲特性 (RQ3)

24 小時評測指標如表 III 所示。下游策略路徑中位延遲為 26.21 μs ，純點陣圖檢查平均僅需 825 ns，CPU 額外負載低於 1.8%。

E. 最小基質極限隔離 (RQ4)

在完全停用 L1、L2、L3 的極限情境下 ($\text{Integrity}(\text{Agent}) = 0 \wedge \text{Integrity}(\text{Upper}) = 0$)，L4 在 906 次敵意嘗試中觀測到 0 次未授權權能轉移，達成 906 次確定性阻斷 (DENY)。

TABLE II
分層消融實驗評測結果 (LAYER-BY-LAYER ABLATION STUDY)

配置	L1	L2	L3	L4	已知 IPI 阻擋	混淆 IPI 阻擋	中位延遲
基準 (無防禦)	N	N	N	N	0.0%	0.0%	0 μ s
+L1 僅語意過濾	Y	N	N	N	85.2%	0.0%	45.2 ms
+L1+L2 密碼學身分	Y	Y	N	N	90.0%	0.0%	25.8 μ s
+L1+L2+L3 拓撲 ABAC	Y	Y	Y	N	93.5%	0.0%	26.0 μ s
完整 DROS (L1-L4)	Y	Y	Y	Y	100.0%	100.0%	26.21 μs
策略誤報率 (FPR)	—	—	—	—	—	—	0 / 22,860

TABLE III
24 小時連續評測效能數據

度量指標	觀測數值
評測請求總量	160,611
下游策略評估延遲 (P50)	26.21 μ s ($\pm$ 0.34 μ s)
下游策略評估延遲 (P99)	242.69 μ s
獨立權能點陣圖檢查延遲	825 ns (平均)
CPU 使用率額外開銷	<1.8% (相較未插樁負載)
策略誤報率 (FPR)	0 / 22,860 (0.0%)
24 小時記憶體洩漏	0 bytes (jemalloc / RSS 剖析)
每角色策略記憶體足跡	<1 MB (4 KB 頁面 / 角色)

F. 與應用層中介軟體的邊界覆蓋對照

我們與微軟代理人治理工具箱 (Microsoft AGT v4.1.0 規範) 進行了跨架構覆蓋比對 (表 IV)。兩者在已宣告工具上皆能提供治理；然而當調用穿透至未受管轄路徑 (原生檔案、原生通訊端、反射) 時，中介軟體難以攔截，而 DROS 於 C-ABI 邊界提供確定性阻斷。

VII. 討論與限制 (DISCUSSION & LIMITATIONS)

DROS 假設工具調用經由受插樁之 C-ABI 分派，不包含針對任意未插樁原生代碼注入或核心層逃逸的防護。正交的系統呼叫層級安全問題 (如勒索軟體與主機特權提升) 則由姐妹作 DROS-PGM [22] 在 OS 系統呼叫邊界處置。

VIII. 結論 (CONCLUSION)

本文介紹了 DROS 四層執行期架構，解決了自主 AI 軟體系統中的代理人-至-執行歸因鴻溝。實證結果證實，確定性二進位執行強制可作為自主 AI 軟體系統的一項穩固軟體架構特性，在認知失陷發生後依然能有效維持權能邊界。

致謝與 AI 協作聲明

依據 IEEE 2024+ 作者指南，作者聲明本文所呈現之核心架構、安全不變式、能力點陣圖模型及實驗設計均由陳濟程 (Chun-Cheng Jimmy Chen) 獨立構思、設計與驗證。生成式 AI 工具僅用於語法修飾、LaTeX 排版與參

考文獻格式整理。本文所述之執行治理技術已申請美國臨時專利保護 (U.S. Provisional Patent Application No. 64/111,973 · Patent Pending)。

REFERENCES

- [1] N. Hardy, “The Confused Deputy: (or why I am not a sympathizer of encryption based security),” *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36–38, Oct. 1988.
- [2] NIST Special Publication 800-207, “Zero Trust Architecture,” National Institute of Standards and Technology, Aug. 2020.
- [3] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications, Version 1.1,” 2023.
- [4] OWASP Foundation, “OWASP Top 10 for Agentic Applications,” 2025.
- [5] MITRE Corporation, “MITRE ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems, Version 4.5,” 2024.
- [6] Y. Zheng, Y. Hu, T. Yu, and A. Quinn, “AgentSight: System-Level Observability for AI Agents Using eBPF,” arXiv preprint arXiv:2508.02736, Aug. 2025.
- [7] Y. Zheng *et al.*, “AgentOps: Enabling Observability of LLM Agents,” arXiv preprint arXiv:2411.05285, Nov. 2024.
- [8] F. Perez and I. Ribeiro, “Ignore Previous Prompt: Attack Techniques For Language Models,” in *Proc. NeurIPS 2022 Workshop on Machine Learning Safety*, Dec. 2022.
- [9] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injections,” arXiv preprint arXiv:2302.12173, Feb. 2023.
- [10] P. Loscocco and S. Smalley, “Integrating Flexible Support for Security Policies into the Linux Operating System,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2001, pp. 29–42.
- [11] S. Smalley and R. Craig, “Security Enhanced (SE) Android: Bringing Flexible MAC to Android,” in *Proc. NDSS Symposium*, Feb. 2013.
- [12] “seccomp(2) – Linux Programmer’s Manual,” Linux Kernel Documentation, Version 6.9, 2024.
- [13] J. B. Dennis and E. C. Van Horn, “Programming Semantics for Multiprogrammed Computations,” *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, Mar. 1966.
- [14] H. M. Levy, *Capability-Based Computer Systems*. Bedford, MA: Digital Press, 1984.
- [15] V. Hu *et al.*, NIST Special Publication 800-162, “Guide to Attribute Based Access Control (ABAC) Definition and Considerations,” NIST, Jan. 2014.
- [16] M. Sporny *et al.*, “Decentralized Identifiers (DIDs) v1.0,” W3C Recommendation, Jul. 2022.
- [17] R. C. Merkle, “A Digital Signature Based on a Conventional Encryption Function,” in *Advances in Cryptology – CRYPTO 1987*, LNCS vol. 293. Berlin: Springer, 1988, pp. 369–378.

TABLE IV
已宣告與未託管調用下之邊界覆蓋對照

架構配置	宣告工具防護	未託管路徑防護	P99 決策延遲
基準 (無治理)	0.0% (失陷)	未監控	0.40 μ s
應用層中介軟體 (AGT v4.x 配置)	100.0% 阻斷	策略攔截缺位	1.80 μ s
C-ABI 執行基質 (DROS GuardVM)	100.0% 阻斷	C-ABI 確定性阻斷	1.20 μ s
雙層縱深防禦 (AGT + DROS)	100.0% 阻斷	C-ABI 確定性阻斷	15.90 μ s

- [18] T. Rebedea *et al.*, “NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails,” arXiv preprint arXiv:2310.10501, 2023.
- [19] H. Inan *et al.*, “Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations,” arXiv preprint arXiv:2312.06674, 2023.
- [20] K. Zhu *et al.*, “PromptBench: Towards Evaluating the Robustness of Large Language Models on Adversarial Prompts,” *IEEE Trans. Knowl. Data Eng.*, 2024.
- [21] European Parliament and Council of the European Union, “Regulation (EU) 2024/1689 Laying Down Harmonised Rules on Artificial Intelligence,” *Official Journal of the European Union*, Jul. 2024.
- [22] C. C. Chen, “Runtime Attribution Framework: An External C-ABI and PKI-Based Zero-Trust Infrastructure for Non-Repudiable Execution Governance in Multi-Agent Systems,” *Zenodo*, DOI: 10.5281/zenodo.20823163, 2026.
- [23] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents,” in *Findings of the ACL 2024*, pp. 3088–3101, 2024.
- [24] F. He *et al.*, “The Emerged Security and Privacy of LLM Agent: A Survey with Case Studies,” *ACM Computing Surveys*, vol. 58, no. 6, Article 162, Dec. 2025.
- [25] A. Birgisson *et al.*, “Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud,” in *Proc. NDSS*, 2014.
- [26] R. Sandhu *et al.*, “Role-Based Access Control Models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [27] Microsoft Corporation, “Agent Governance Toolkit: Deterministic Governance and Policy Enforcement for Autonomous AI Agents,” 2026. [Online]. Available: <https://github.com/microsoft/agent-governance-toolkit>